ספר פרויקט

משחק דמקה ב־Java ובּ־JavaFX

תיעוד תכנון, מימוש והפעלה

משחק רשת TCP, חשבונות ופרופיל משתמש

מגישים:

רביע אבו עאד — ת"ז 213984214

מותנא פרחאת — ת"ז 325079143

מנחה: ענבל רונן

אוקטובר 2026

תקציר ותוכן עניינים

הפרויקט הוא משחק דמקה ב־Java וב־JavaFX, הכולל משחק מול BOT ומשחק בין שני שחקנים דרך שרת TCP. השרת מנהל חדרים, מאמת מהלכים ושומר חשבונות ותוצאות ב־SQLite. לכל משתמש מחובר יש פרופיל המציג סטטיסטיקה והיסטוריית משחקים.

1. מבוא, מטרות ודרישות 3
2. כללי המשחק ומימוש מהלך 4
3. מבנה המערכת ואחריות המחלקות 5
4. תקשורת TCP וניהול חדרים 7
5. השחקן הממוחשב — BOT 9
6. חשבונות, SQLite ופרופיל 10
7. ממשק המשתמש ומסכי המערכת 12
8. בדיקות ואימות המערכת 14
9. הפעלה ומדריך משתמש 15
10. שימוש בכלי AI והתרומה האישית 16
11. סיכום, אתגרים ומקורות 17

1. מבוא, מטרות ודרישות

1.1 רקע ומטרת הפרויקט

דמקה היא משחק שבו ההחלטה על מהלך תלויה במיקום הכלים, באפשרויות האכילה ובתגובת היריב. מטרת הפרויקט היא ליצור יישום דמקה בעל ממשק גרפי, המאפשר משחק מול שחקן אחר באמצעות שרת ומשחק מול BOT. כל שחקן מפעיל את היישום במחשב שלו ומתחבר לחדר משותף.

1.2 מטרות

- להציג לוח ברור, בחירת כלי ומשבצת יעד, אנימציה ומשוב על מהלך שגוי.
- לאפשר יצירת חדר, שיתוף קוד, הצטרפות וסנכרון בין שני שחקנים.
- להוסיף הרשמה, התחברות ופרופיל עם סטטיסטיקה והיסטוריה שנשמרות לאורך זמן.

1.3 היקף

המערכת כוללת משחק בין שני שחקנים ברשת מקומית, משחק מול BOT ופרופיל אישי עם סטטיסטיקה והיסטוריה. משתמש מחובר יכול ליצור חדר או להצטרף לחדר. אפשר גם לתרגל מול BOT ללא חשבון, ללא רישום תוצאות בפרופיל.

1.4 תוצר וסביבת פיתוח

התוצר כולל יישום גרפי לשחקנים, שרת לניהול המשחק ומסד נתונים לשמירת הפרופילים. המשחק כתוב ב־Java, הממשק נבנה באמצעות JavaFX והבנייה מנוהלת ב־Gradle. Main מפעילה את יישום השחקן ו־DamkaServer מפעילה את השרת. העיצוב נמצא בקובץ damka.css.

2. כללי המשחק ומימוש מהלך

2.1 מצב פתיחה ותנועה

הלוח בגודל 8×8, עם 12 כלים לכל שחקן על המשבצות הכהות. X מתחיל בתחתית, נע מעלה ופותח את המשחק; O מתחיל למעלה ונע מטה. הקואורדינטות הן בטווח 0 עד 7. כלי רגיל נע משבצת אחת באלכסון קדימה, ואוכל בקפיצה של שתי משבצות מעל יריב אל יעד פנוי. כלי רגיל אוכל קדימה בלבד.

2.2 אכילה מחייבת והכתרה

אם קיימת אכילה, צעד רגיל אינו מותר. כאשר אפשר לאכול שוב אחרי קפיצה, ממשיכים עם אותו כלי ואותו שחקן נשאר בתור. כלי x בשורה 0 וכלי o בשורה 7 מוכתרים למלכים. מלך נע ואוכל לשני הכיוונים, אך אינו מעופף: צעד של משבצת אחת או קפיצת אכילה של שתי משבצות. ההכתרה נעשית לפני בדיקת ההמשך, ולכן מלך חדש עשוי להמשיך לאכול לאחור.

2.3 Game היא הסמכות לכללים

getValidMoves מחזירה את האפשרויות החוקיות: אכילות בלבד כאשר קיימת אכילה, או צעדים רגילים אחרת. באמצע רצף היא מגבילה את האפשרויות למיקום captureContinuationPosition. Game.move מחפשת את המקור והיעד ברשימה; מהלך שנדחה אינו משנה את הלוח או התור.

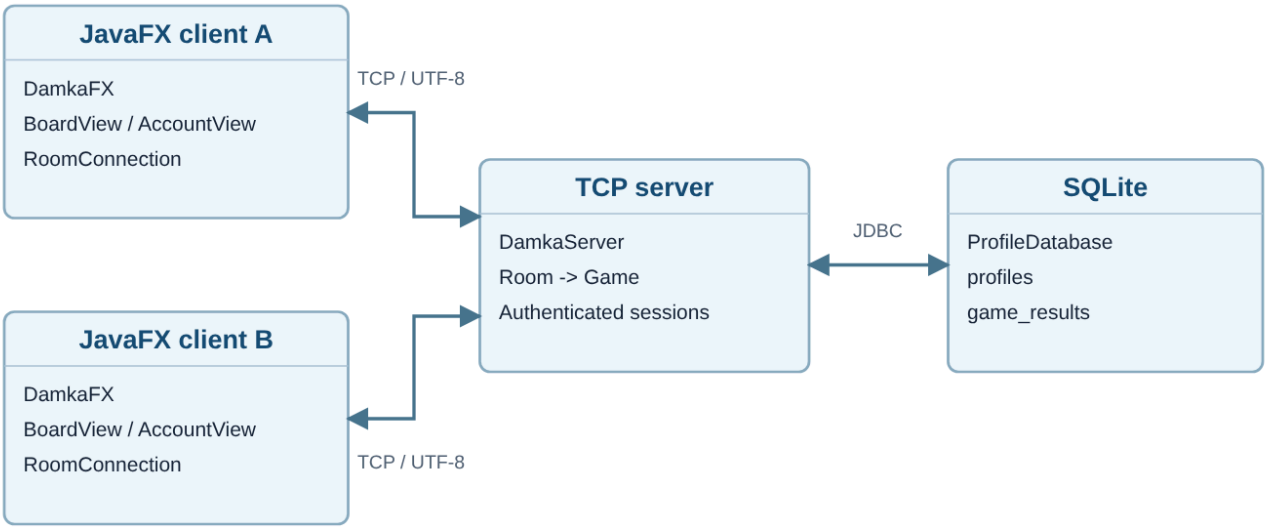
אחרי אישור Game מסירה כלי שנאכל, מזיזה את הכלי, מכתירה, שומרת lastMove ומשאירה חובת המשך או מחליפה תור. Game.Move כוללת מקור, יעד, סימן כלי ומיקום אכילה לצורך תצוגה. Board מבצעת גישה והזזה של הנתונים, בלי להחליט על חוקיות.

2.4 ניצחון

getWinnerSymbol מחזירה X או O כאשר ליריב אין מהלך חוקי, ו־'\0' כאשר אין מנצח. היעדר כלים גורם גם להיעדר מהלכים. פרישה, עזיבה וניתוק במשחק רשת מטופלים ב־Room, ולא כחוק תנועה של Game.

3. מבנה המערכת ואחריות המחלקות

3.1 שרת, לקוחות ושמירה



איור 1 — כל שחקן מפעיל לקוח; מצב המשחק המקוון והמסד נמצאים בשרת.

DamkaFX מנהלת קלט, מצב מסך ותגובות להודעות. BoardView מציירת ומנפישה, AccountView מציגה הזדהות ו־ProfileWindow מציגה פרופיל. RoomConnection מבצעת תקשורת ברקע. DamkaServer מקבלת לקוחות ומנתבת בקשות. Room מחזיקה משחק מקוון ושני משתתפים, ו־ProfileDatabase קוראת וכותבת SQLite.

3.2 שני מצבי משחק

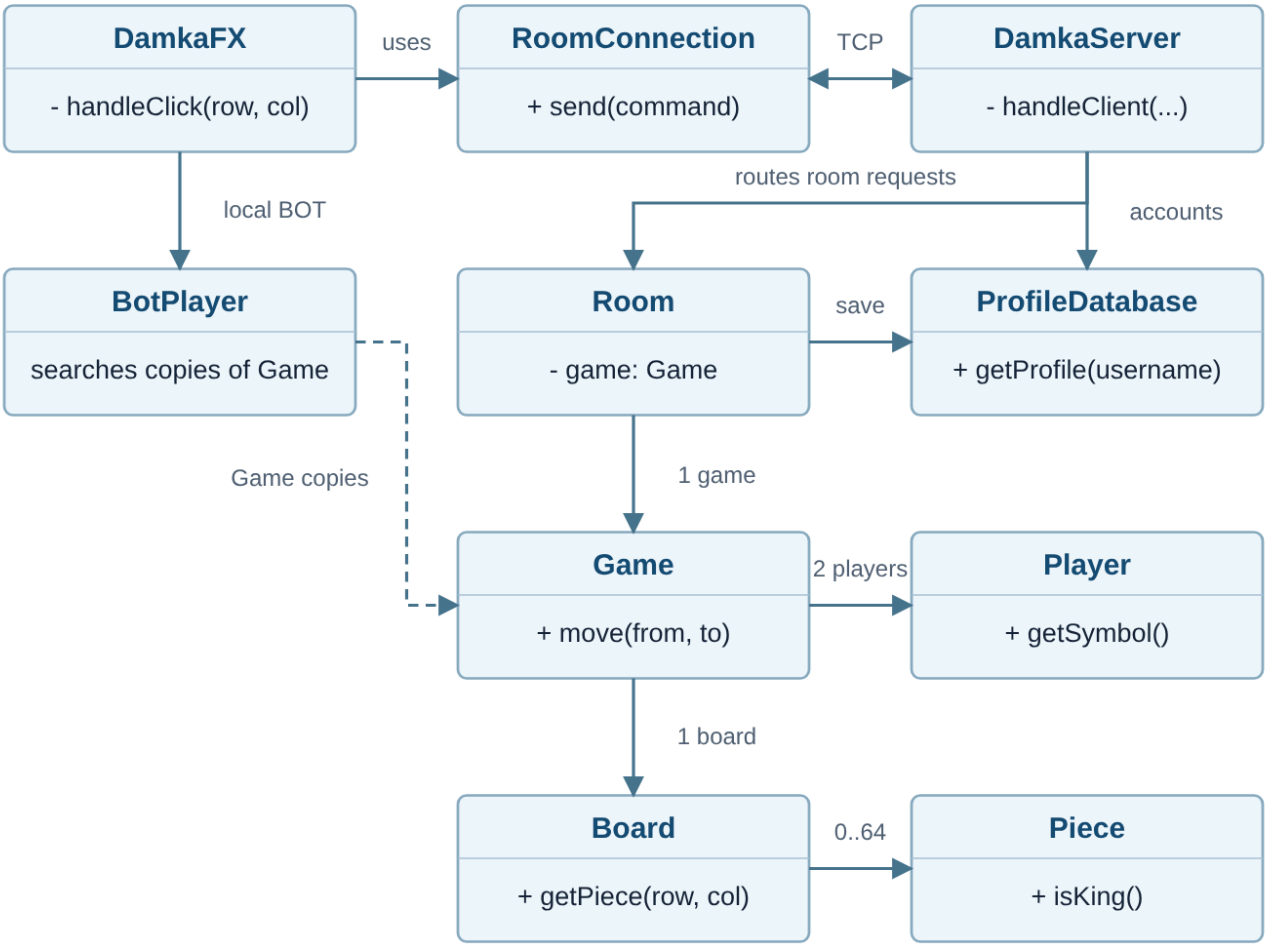
מול אדם אחר Game נמצאת בתוך Room בשרת, והלקוח מחזיק תמונת לוח לציור בלבד. מול BOT קיימת botGame מקומית ב־DamkaFX. BotPlayer מחפשת מהלך על עותקי Game. בשני המצבים משתמשים באותם כללים ובאותו BoardView.

3.3 הפרדה בין סוגי בדיקות

Game בודקת תנועה, אכילה, הכתרה וניצחון. Room בודקת מי שלח בקשה, חדר פעיל, תור ומבנה נתונים. הממשק בודק בחירה וזמינות פעולה. ההפרדה מונעת שכפול של כללי המשחק ומאפשרת בדיקה ללא חלון גרפי.

3. מבנה המערכת ואחריות המחלקות — המשך

3.4 תרשים מחלקות



איור 2 — הקשרים המרכזיים בין הממשק, התקשורת, המשחק והשמירה.

Position שומרת שורה ועמודה; Piece שומרת סימן, ואות גדולה מציינת מלך. Player היא שחקן במודל המשחק ואינה חשבון במסד. ProfileData היא רשומת נתוני פרופיל להעברה לתצוגה. החץ TCP מייצג קשר בין תהליכים, ולא קריאה ישירה למתודה על אובייקט משותף. התרשים מציג את הקשרים המרכזיים, ולא את כל מחלקות הספריות.

4. תקשורת TCP וניהול חדרים

4.1 חיבור השחקנים לשרת

כל שחקן מפעיל לקוח JavaFX שמתחבר לשרת באמצעות TCP. ServerSocket ממתינה לחיבורים חדשים, ו־Socket משמשת לשליחה ולקבלה של הודעות. החיבור נפתח בעת הרשמה או כניסה ונשאר פעיל גם במהלך המשחק ובצפייה בפרופיל. הודעות הרשת מטופלות ברקע כדי שהממשק יוכל להמשיך להגיב לפעולות המשתמש.

4.2 יצירת חדר והצטרפות

חדר מחבר בין שני משתתפים למשחק אחד. היוצר מקבל קוד בן שישה תווים ומשחק בתור X. שחקן אחר מזין את הקוד, מצטרף ומשחק בתור O. השרת מאשר את ההצטרפות, מודיע לשניהם שהחדר מוכן ושולח את לוח הפתיחה ואת התור הראשון.

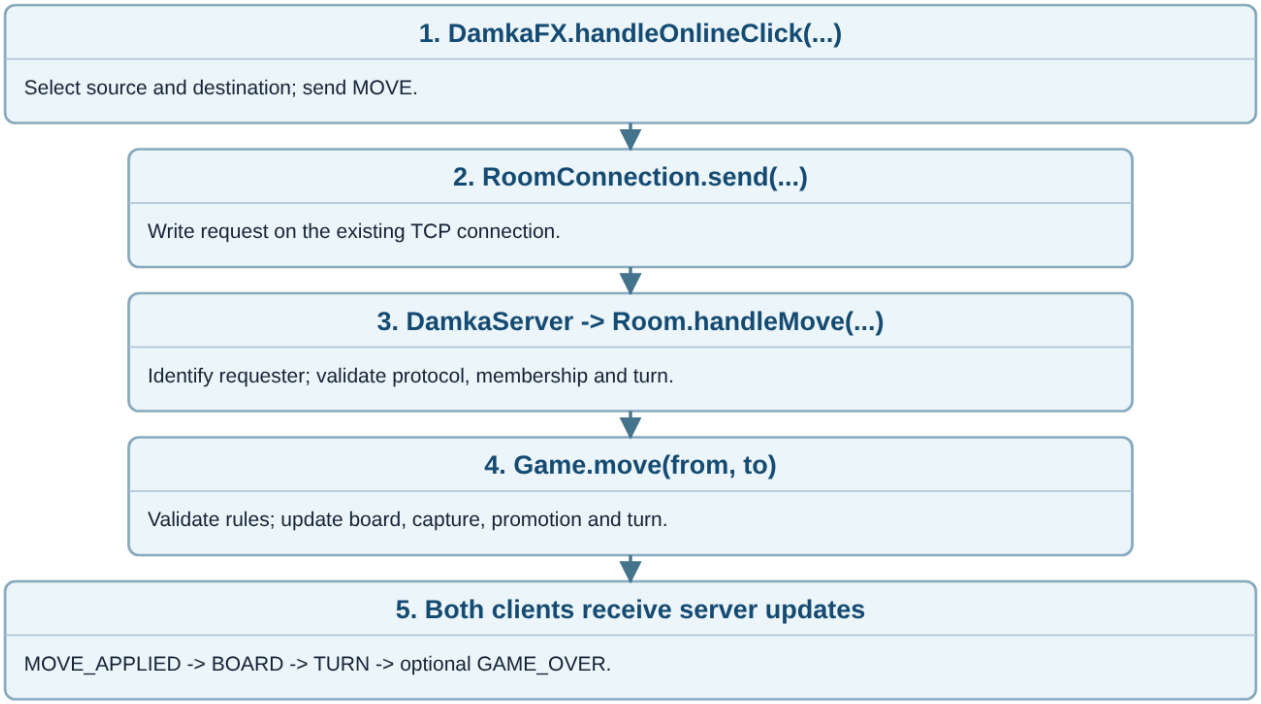
בקשה או אירוע	תגובה מרכזית
CREATE_ROOM	ROOM_CREATED <code>, PLAYER X
JOIN_ROOM <code>	ROOM_JOINED <code>, PLAYER O
השחקן השני התקבל	ROOM_READY, BOARD, TURN X לשניהם
MOVE <fromRow> <fromCol> <toRow> <toCol>	MOVE_APPLIED, BOARD, TURN או ERROR
RESIGN / LEAVE_ROOM	GAME_OVER / ROOM_LEFT לפי מצב החדר

4.3 העברת מצב הלוח

מצב הלוח מועבר כמחרוזת של 64 סימנים לפי סדר השורות. נקודה מייצגת תא ריק, x/o כלי רגיל ו־X/O מלך. הלקוחות משתמשים במחרוזת כדי לצייר את הלוח. השרת הוא שקובע את מצב המשחק, ושני המשתתפים מקבלים ממנו את אותה תמונה.

4. תקשורת TCP וניהול חדרים — המשך

4.4 מהלך אחד מתחילתו ועד התצוגה



איור 3 — מסלול הבקשה והעדכון לשני הלקוחות.

המשתמש בוחר כלי ומשבצת יעד. הלקוח שולח בקשת MOVE, והשרת בודק את המשתתף, התור וחוקיות המהלך באמצעות Game. מהלך מאושר מעדכן את המשחק ונשלח לשני הלקוחות; בקשה לא חוקית משאירה את הלוח כפי שהיה.

4.5 תצוגה מסונכרנת

שני הלקוחות מקבלים את פרטי המהלך, את הלוח המעודכן ואת התור הבא. תחילה מוצגת אנימציית התנועה, ואחריה תמונת הלוח החדשה. כך שני השחקנים רואים את אותה התקדמות במשחק.

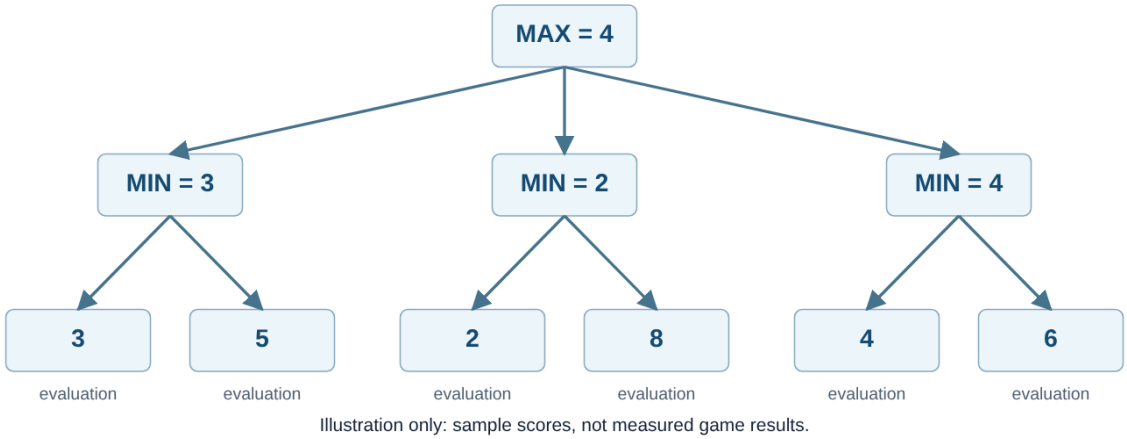
4.6 סיום משחק

המשחק מסתיים בניצחון לפי הכללים או בפרישת משתתף. עזיבה או ניתוק במהלך משחק פעיל מעניקים ניצחון ליריב. השרת שולח את התוצאה ושומר אותה בפרופילים. עזיבת חדר לפני הצטרפות שחקן שני אינה נרשמת כמשחק שהסתיים. לאחר הסיום ניתן לפתוח משחק נוסף.

5. השחקן הממוחשב — BOT

5.1 בחירת מהלך באמצעות Minimax

BotPlayer בוחנת מהלכים חוקיים ואת תגובות היריב בעזרת Minimax. בתור BOT נבחר הציון הגבוה ביותר, ובתור היריב הציון הנמוך ביותר. הבדיקה נעשית על עותקי Game, ולכן אינה משנה את הלוח שמוצג. לאחר הבחירה מבוצע המהלך במשחק המקומי.



איור 4 — דוגמה: MAX בוחרת את התוצאה 4 מתוך תוצאות MIN.

5.2 רמות קושי

רמה	עומק	השפעה
Easy	1	בודקת את תוצאת המהלך המועמד.
Medium	2	בודקת שכבה נוספת של מהלכים.
Hard	3	בודקת יותר אפשרויות, עם יותר עבודה.

בכל הרמות פועל אותו אלגוריתם. ההבדל הוא מספר שכבות המהלכים שה־BOT בוחנת.

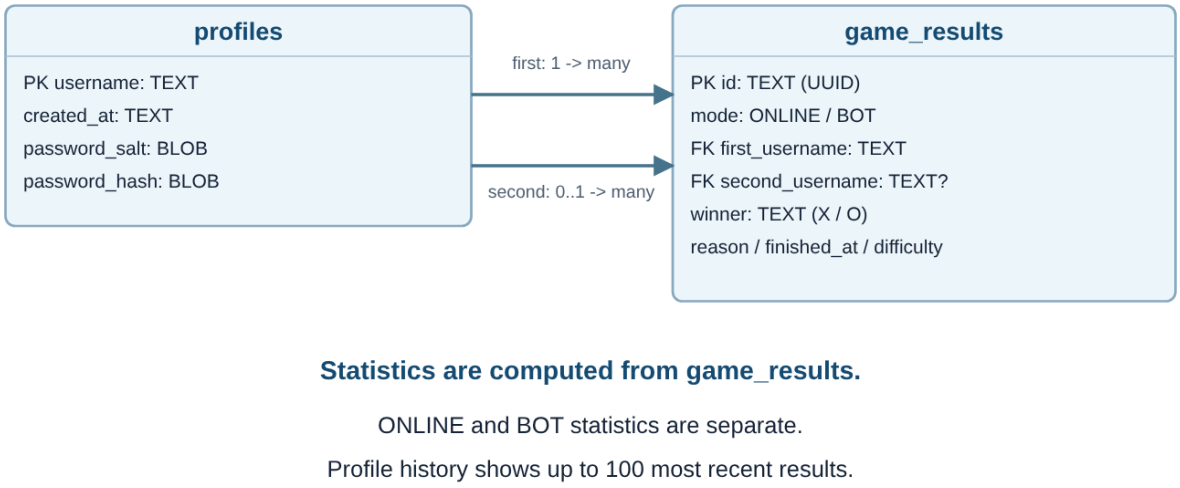
5.3 הערכת מצב הלוח

כלי רגיל מקבל 3 נקודות ומלך 6, משום שמלך יכול לנוע ולאכול לשני הכיוונים. ההערכה משווה את הכלים של BOT לכלי היריב. האדם משחק X וה־BOT משחק O; שני הצדדים מבצעים מהלכים לפי אותם כללי Game, כולל המשך אכילה עם אותו כלי.

6. חשבונות, SQLite ופרופיל

6.1 נתונים מתמשכים בשרת

ProfileDatabase שומרת את החשבונות ואת תוצאות המשחקים במסד SQLite שנמצא במחשב השרת. הנתונים נשארים גם אחרי סגירת היישום והפעלה מחדש, ולכן המשתמש יכול לחזור לחשבון ולראות את הפרופיל שלו.



איור 5 — profiles מחזיקה חשבונות, ו־game_results מחזיקה תוצאות.

6.2 סכמה

טבלה	שדות עיקריים
profiles	username (PK), created_at, password_salt, password_hash
game_results	id (PK), mode, first_username / second_username (FK), winner, reason, finished_at, difficulty

6.3 הרשמה והתחברות

בהרשמה המשתמש בוחר שם וסיסמה ומאשר את הסיסמה. השרת בודק את הפרטים ויוצר חשבון כאשר השם פנוי. הסיסמה נשמרת בצורת hash ולא כטקסט המקורי. בעת כניסה השרת מאמת את הפרטים ומאפשר למשתמש לגשת למשחק ולפרופיל.

לאחר הכניסה כל פעולה בחיבור משויכת לחשבון שאומת. המשתמש יכול ליצור חדר, להצטרף למשחק ולצפות בתוצאות שלו. Sign out מסיימת את ההתחברות ומאפשרת להיכנס לחשבון אחר.

6. חשבונות, SQLite ופרופיל — המשך

6.4 שמירה פעם אחת

עם הצטרפות השחקן השני נוצר matchId מסוג UUID. בסיום משחק מקוון נשמרת שורה אחת עם המשתתפים, המנצח, הסיבה וזמן הסיום. מזהה id ייחודי מונע ספירה כפולה: חזרה על אותה תוצאה אינה מוסיפה משחק, ושימוש באותו id לתוצאה אחרת נדחה. חדר המתנה שנסגר אינו נרשם.

6.5 סטטיסטיקה והיסטוריה

getProfile קוראת את התוצאות שבהן החשבון הוא המשתתף הראשון או השני. לפי המושב ו־winner נקבע Win או Loss. Online ו־BOT נספרים בנפרד. Played, Wins ו־Losses כוללים את כל התוצאות; ההיסטוריה ממוינת מהחדש לישן ומציגה עד 100 משחקים. שניהם מחושבים מאותה קריאה למסד.

בקשה	תשובה ומשמעות
SIGNUP / LOGIN	AUTH_OK או AUTH_ERROR עבור פרטי החשבון.
PROFILE_REQUEST	PROFILE עם ProfileData מקודדת, או PROFILE_ERROR.
BOT_RESULT <id> <WIN|LOSS> <difficulty> <time>	BOT_SAVED או BOT_ERROR. זהות החשבון נלקחת מהחיבור.

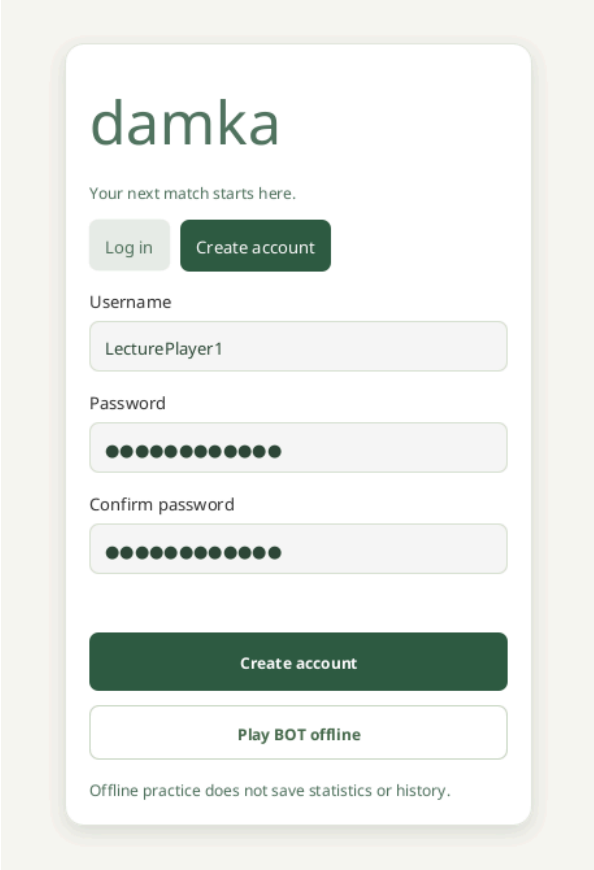
ProfileWindow מציגה את הנתונים. פתיחת החלון או Refresh שולחות PROFILE_REQUEST בלי שם משתמש לבחירה. השרת קורא את הפרופיל של החשבון המאומת, והלקוח מפענח באמצעות ProfileData.decode. סגירת חלון הפרופיל משאירה את חלון המשחק פתוח.

6.6 מתי תוצאת BOT נשמרת

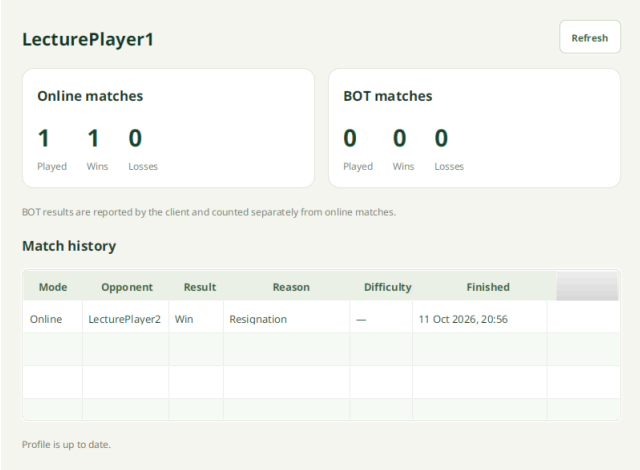
תוצאת BOT נשמרת כאשר המשחק מתחיל ומסתיים באותו חשבון פעיל ובאותו חיבור. המשחק מתנהל במחשב השחקן, והלקוח מדווח לשרת את התוצאה; היא מוצגת בנפרד ממשחקי הרשת. משחק ללא התחברות הוא אימון שאינו נכנס להיסטוריה, גם לאחר כניסה מאוחרת לחשבון.

7. ממשק המשתמש ומסכי המערכת

7.1 הזדהות ופרופיל



איור 6 — הרשמה בממשק הנוכחי.



איור 7 — פרופיל אחרי משחק הדגמה.

AccountView מציעה Log in, Create account ו־Play BOT offline. הצלחת הזדהות מובילה למסך המשחק. Profile פותח חלון עם שתי קבוצות סטטיסטיקה וטבלת היסטוריה; Refresh מבקש מידע חדש. הצילומים הופקו עם חשבונות הדגמה במסד זמני, ולא עם נתוני שחקנים אמיתיים.

7.2 בחירה, משוב ועיצוב

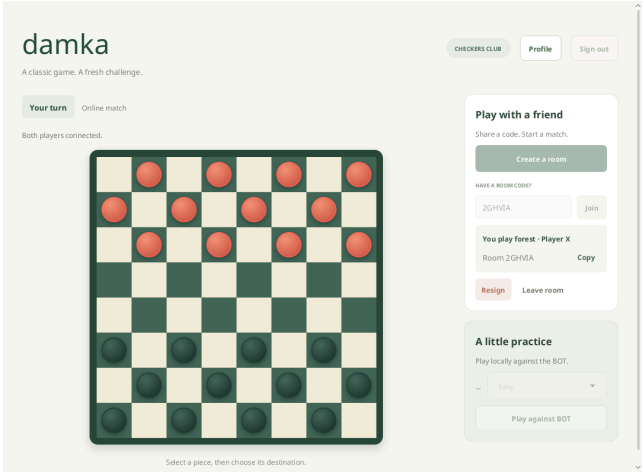
לחיצה על כלי בוחרת אותו, ולחיצה על יעד פנוי מבקשת מהלך. BoardView מציגה לוח קרם וירוק, כלי X בירוק וכלי O באלמוג. בחירה מסומנת בטבעת זהובה ומלך במסגרת עבה. שגיאה מופיעה בכרטיס צף שניתן לסגור; היא אינה מקטינה את הלוח, ופעולה מוצלחת מנקה הודעה קודמת. העיצוב נמצא ב־damka.css.

7.3 חלון גמיש

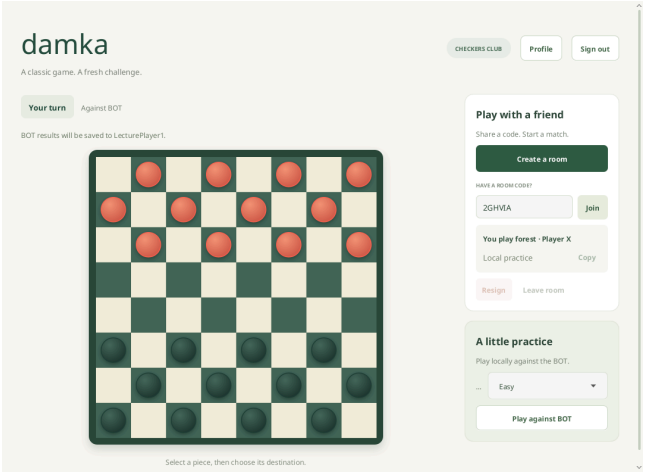
הלוח מתאים את גודלו לחלון. בחלון צר כרטיסי המשחק וה־BOT עוברים מתחתיו, וקיימת גלילה כשצריך. קוד החדר ניתן להעתקה באמצעות Copy. כתובת השרת והפורט מוגדרים בהפעלה ואינם שדות במסך.

7. ממשק המשתמש ומסכי המערכת — המשך

7.4 משחק רשת ומשחק BOT



איור 8 — חדר פעיל מול שחקן.



איור 9 — משחק BOT מקומי.

Create a room ו־Join משתמשות בחיבור המאומת. אחרי ששני המשתתפים מחוברים מוצגים הלוח והתור של השרת. Resign ו־Leave room מטפלות בסיום. Play against BOT מתחיל משחק מקומי; בוחרים Easy, Medium או Hard. הממשק מונע מהלך נוסף בזמן בקשה או אנימציה.

7.5 אותה אנימציה בשני המצבים

BoardView.animateMove מסתירה את הכלי במקור ומציגה כלי נע בשכבה מעל הלוח. Timeline מנפישה במשך כ־320 מילישניות; FadeTransition מעלימה כלי שנאכל. מרכזי התאים מחושבים בכל פריים כדי ששינוי גודל וגלילה לא יזיזו את מסלול התנועה. בסוף מוצגת התמונה המעודכנת. אותו רכיב משמש BOT ומשחק רשת.

7.6 היקף ההדגמה

הצילומים מציגים את זרימת השימוש ביישום: הזדהות, יצירת חדר, הצטרפות, ביצוע מהלכים וסיום משחק. בפרופיל מוצגת תוצאת משחק ההדגמה שנשמרה בשרת.

8. בדיקות ואימות המערכת

8.1 בדיקות אוטומטיות

הבדיקות האוטומטיות משתמשות ב־JUnit 5 ומכסות את כללי המשחק, תקשורת TCP, החשבונות ושמירת הנתונים. הטבלה מרכזת את תחומי הבדיקה:

מחלקה	בדיקות	מה נבדק
GameRulesTest	11	תנועה, אכילה מחייבת ורצף, הכתרה, העתקה וניצחון.
RoomProtocolTest	11	מהלכים, סנכרון, הצטרפות מתחרה, סיום ושמירת תוצאה.
RoomConnectionTest	4	התחברות, סדר הודעות, תגובות וסגירה.
AuthProfileTest	14	הרשמה, כניסה, זהות, חדרים, פרופיל ושמירת BOT.
ProfileDatabaseTest	7	התמדה, סיסמאות, תוצאות ללא כפילות והיסטוריה.

8.2 תוצאות

בהרצה המתועדת מ־11.10.2026 עברו כל 47 הבדיקות והבנייה הסתיימה בהצלחה. נבדקו בין היתר אכילה מחייבת, רצף אכילות, הכתרה, תורות, הרשמה, כניסה ושמירת תוצאות ללא כפילות.

9. הפעלה ומדריך משתמש

9.1 התחלת משחק ברשת

- פותחים את הפרויקט בסביבת הפיתוח ומפעילים את DamkaServer.
- כל שחקן מפעיל את Main ונרשם או נכנס לחשבון. לצורך הדגמה במחשב אחד פותחים שני חלונות.
- השחקן הראשון בוחר Create a room ומשתף את הקוד. השני מזין אותו ובוחר Join.
- כאשר שני השחקנים מחוברים, X מתחיל. בוחרים כלי ומשבצת יעד וממתינים לעדכון התור.

אפשר להפעיל את המחלקות באמצעות Run ב־VS Code לאחר ייבוא הפרויקט. במשחק בין שני מחשבים הם מחוברים לאותה רשת מקומית, ושני הלקוחות מוגדרים לכתובת ולפורט של השרת.

9.2 פרופיל וסיום משחק

Profile פותח את הסטטיסטיקה וההיסטוריה. Refresh מעדכן את הנתונים, וסגירת חלון הפרופיל מחזירה לחלון המשחק. Resign מאפשר לפרוש ו־Leave room לצאת מהחדר. לאחר הסיום אפשר ליצור משחק נוסף.

9.3 משחק מול BOT

Play against BOT מתחיל משחק מקומי ברמת Easy, Medium או Hard. לתרגול ללא שרת מפעילים רק Main ובוחרים Play BOT offline. משחק אימון אינו נרשם בפרופיל.

10. שימוש בכלי AI והתרומה האישית

10.1 היקף ואופן השימוש

נעשה שימוש נרחב ב־Codex וב־GitHub Copilot ללמידה, תכנון, יצירת קוד, עיצוב, בדיקות ותיעוד. הסיוע כלל מימוש תקשורת TCP, חיבור JavaFX לשרת, ניהול חדרים, חשבונות, SQLite והפרופיל.

10.2 תרומה אישית בתהליך

מותנא הגדיר דרישות וקיבל החלטות על התנהגות המערכת, הוביל את הפיתוח בעזרת הכלים ובדק את היישום ב־Windows. הוא נעזר בהסברים על Socket וזרימת ההודעות, קרא את הקוד ובחן את התאמת ההצעות לדרישות. התהליך כלל הבנת הקשרים בין המחלקות ובדיקת פעולות המשתמש.

10.3 דוגמה מעשית לעבודה עם AI

בעת עיצוב המשוב למשתמש, מותנא הגדיר שהודעות שגיאה יוצגו בצורה ברורה בלי לשנות את גודל הלוח. בעזרת AI מומש כרטיס הודעה צף. מותנא בחן את אופן הצגתו בזמן המשחק וביקש התאמות לעיצוב ולנוחות השימוש. התהליך כלל הגדרת דרישה, שימוש בכלי לסיוע במימוש ובחינת התוצאה ביישום.

11. סיכום, אתגרים ומקורות

11.1 סיכום

הפרויקט מממש משחק דמקה במבנה שרת ולקוחות, עם חדרי משחק, חשבונות ופרופילים. השרת מנהל את המשחק המקוון, Game מרכזת את הכללים והלקוחות מציגים את המצב ומבקשים פעולות. אפשר לשחק גם מול BOT ולצפות בסטטיסטיקה ובהיסטוריה של החשבון.

11.2 אתגרים ופתרונות

- לימוד כללי הדמקה: לפני מימוש הכללים נלמדו התנועה, האכילה המחייבת, רצף האכילות וההכתרה, והוגדרו מצבי משחק לבדיקת המימוש.
- עבודה עם JavaFX: נדרשה היכרות עם ספריית ממשק חדשה, ארגון רכיבים בחלון וחיבור פעולות לכפתורים וללוח.
- יישום TCP: הידע התאורטי מקורס רשתות תקשורת יושם לראשונה בחיבור מעשי בין לקוחות לשרת ובהעברת הודעות משחק.
- ממשק יישום לעומת HTML: הניסיון בבניית דפי HTML שימש בסיס לחשיבה על עיצוב, והעבודה ב־JavaFX דרשה ללמוד כיצד לארגן רכיבי Java ולהגיב לאירועי משתמש.

11.3 המשך פיתוח

אפשר להוסיף צ׳אט בין המשתתפים בתוך חדר המשחק. הודעות הצ׳אט יוכלו לעבור דרך חיבורי TCP הקיימים אל השרת, שיעביר אותן לשחקן האחר באותו חדר. התוספת תאפשר לשחקנים להתכתב במהלך המשחק, לצד הלוח והתור.

11.4 מקורות

מאגר הפרויקט ב־GitHub:

https://github.com/MuthannaFrhat/damka-game